

Django Attendance System

U You 5/27/2026, 11:23:38 AM

i want to learn to make a django minor project.

There are models:

1. student(id,sname,[course.id](#))
2. faculty(id,fname)
3. course(id,cname)
3. academics(id,[course.id](#),sname)
- 5.attendance([student.id](#),[faculty.id](#),[course.id](#),[academics.id](#),timestamp)

remark on what is the level of normalizzation.

now then make the templates for the faculty to make attendance using django forms.

properties of the form:

faculty open the template for make attendance.

There is dropdown for subject, and the course.

Then there is a single page list containing circular divs : initially red for unmarked attendance and green on selected attendance and all showing the id of the students. and the attendance of the selected students must be got marked.

This must be in a form and there must be a button to mark attendance, and we could add attendance once in a day.

the student could see the attendance for the sujet as : number of classes, classes attended and then the percentage of attendance.

AI ChatGPT 5/27/2026, 11:23:39 AM

Your schema is roughly in **2NF**, but not fully normalized.

Problems in current schema

academics(id, course.id, sname)

- sname should not be stored.
- Student name already exists in student.
- Causes redundancy.

Better:

```
Academics(id, student_id, course_id)
```

attendance(student.id, faculty.id, course.id, academics.id, timestamp)

Problems:

- course.id can already come from academics
- academics.id already identifies student-course relation
- Too much duplication.

Better structure:

```
Attendance(  
    id,  
    academics_id,  
    faculty_id,  
    date,  
    status  
)
```

or simpler:

```
Attendance(  
    id,  
    student_id,  
    course_id,  
    faculty_id,  
    date,  
    present  
)
```

Add:

```
unique_together = ('student', 'course', 'date')
```

This prevents multiple attendance in one day.

Recommended Final Models

```
class Course(models.Model):  
    cname = models.CharField(max_length=100)
```

```
class Faculty(models.Model):  
    fname = models.CharField(max_length=100)
```

```
class Student(models.Model):  
    sname = models.CharField(max_length=100)  
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
```

```
class Attendance(models.Model):  
    student = models.ForeignKey(Student, on_delete=models.CASCADE)  
    faculty = models.ForeignKey(Faculty, on_delete=models.CASCADE)  
    course = models.ForeignKey(Course, on_delete=models.CASCADE)  
  
    date = models.DateField(auto_now_add=True)  
  
    present = models.BooleanField(default=False)  
  
class Meta:  
    unique_together = ('student', 'course', 'date')
```

This is near 3NF.

Faculty Attendance UI Design

Workflow

Faculty opens page.

Top Section

Dropdowns:

- Course
- Subject

Button:

- Load Students
-

Student Grid UI

After loading students:

Display circular buttons/cards.

Example:

```
<div class="student-circle absent">
  101
</div>
```

CSS:

- Red = absent/unselected
- Green = selected/present

On click:

- Toggle class
 - Hidden checkbox updated
-

Suggested UI

```
image_group["layout":"carousel","aspect_ratio":"16:9","query":
```

```
“modernattendancemanagementUIcircularstudentselection”, “djangoattende
,“num_per_query”:1}
```

Django Form Structure

Main form

```
class AttendanceForm(forms.Form):  
    course = forms.ModelChoiceField(queryset=Course.objects.all())
```

Template Logic

```
<form method="POST">  
  
    {{ form.course }}  
  
    <div class="student-container">  
  
        {% for student in students %}  
  
            <div class="student-circle"  
                onclick="toggleAttendance(this)">  
  
                {{ student.id }}  
  
                <input type="checkbox"  
                    name="students"  
                    value="{{ student.id }}"  
                    hidden>  
            </div>  
  
        {% endfor %}  
  
    </div>  
  
    <button type="submit">  
        Mark Attendance  
    </button>  
  
</form>
```

JS Toggle

```
function toggleAttendance(el){  
  
  el.classList.toggle("present");  
  
  let checkbox = el.querySelector("input");  
  
  checkbox.checked = !checkbox.checked;  
}
```

CSS

```
.student-circle{  
  width:70px;  
  height:70px;  
  
  border-radius:50%;  
  
  background:red;  
  color:white;  
  
  display:flex;  
  justify-content:center;  
  align-items:center;  
  
  cursor:pointer;  
}  
  
.student-circle.present{  
  background:green;  
}
```

Attendance Save Logic

View

```
from datetime import date

def mark_attendance(request):

    if request.method == "POST":

        selected_students = request.POST.getlist("students")

        course_id = request.POST.get("course")

        for sid in selected_students:

            Attendance.objects.get_or_create(

                student_id=sid,
                course_id=course_id,
                date=date.today(),

                defaults={
                    'faculty': request.user.faculty,
                    'present': True
                }
            )
```

Student Attendance Report

Formula

```
total_classes = Attendance.objects.filter(
    course=course
).values('date').distinct().count()

attended = Attendance.objects.filter(
    student=student,
    course=course,
```

```
present=True
).count()

percentage = (attended / total_classes) * 100
```

Student Dashboard UI

Subject	Total Classes	Attended	Percentage
Java	40	36	90%

Technologies You Will Learn

This project teaches:

- Django Models
- Foreign Keys
- Django Forms
- Querysets
- Aggregation
- Template rendering
- JavaScript DOM
- Attendance logic
- Constraints
- CRUD
- Authentication

This is a very good **minor project level** system.

U You 5/27/2026, 11:24:51 AM

thankyou for the above,
I want to make all above from scratch under your assistance, so do it.
make a project , and then make the suitable apps and make the code so that i could
acheive it.

AI ChatGPT 5/27/2026, 11:24:52 AM

Step 1 — Create Django Project

Open terminal.

```
django-admin startproject attendanceSystem
```

Go inside:

```
cd attendanceSystem
```

Create apps:

```
python manage.py startapp appStudent  
python manage.py startapp appFaculty  
python manage.py startapp appCourse  
python manage.py startapp appAttendance
```

Final Structure

```
attendanceSystem/  
  
    attendanceSystem/  
  
        appStudent/  
        appFaculty/  
        appCourse/  
        appAttendance/  
  
        manage.py
```

Step 2 — Register Apps

Open:

```
attendanceSystem/settings.py
```

Add:

```
INSTALLED_APPS = [  
  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'appStudent',  
    'appFaculty',  
    'appCourse',  
    'appAttendance',  
]
```

Step 3 — Make Models

appCourse/models.py

```
from django.db import models  
  
class Course(models.Model):  
  
    cname = models.CharField(max_length=100)  
  
    def __str__(self):  
        return self.cname
```

appFaculty/models.py

```
from django.db import models

class Faculty(models.Model):

    fname = models.CharField(max_length=100)

    def __str__(self):
        return self.fname
```

appStudent/models.py

```
from django.db import models
from appCourse.models import Course

class Student(models.Model):

    sname = models.CharField(max_length=100)

    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE
    )

    def __str__(self):
        return self.sname
```

appAttendance/models.py

```
from django.db import models

from appStudent.models import Student
from appFaculty.models import Faculty
from appCourse.models import Course
```

```
class Attendance(models.Model):

    student = models.ForeignKey(
        Student,
        on_delete=models.CASCADE
    )

    faculty = models.ForeignKey(
        Faculty,
        on_delete=models.CASCADE
    )

    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE
    )

    date = models.DateField(auto_now_add=True)

    present = models.BooleanField(default=False)

    class Meta:

        unique_together = (
            'student',
            'course',
            'date'
        )

    def __str__(self):
        return f"{self.student} - {self.date}"
```

Step 4 — Make Migrations

Run:

```
python manage.py makemigrations
```

Then:

```
python manage.py migrate
```

Step 5 — Register Admin

appCourse/admin.py

```
from django.contrib import admin
from .models import Course

admin.site.register(Course)
```

appFaculty/admin.py

```
from django.contrib import admin
from .models import Faculty

admin.site.register(Faculty)
```

appStudent/admin.py

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

appAttendance/admin.py

```
from django.contrib import admin
from .models import Attendance

admin.site.register(Attendance)
```

Step 6 — Create Superuser

```
python manage.py createsuperuser
```

Step 7 — Run Server

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/admin
```

Add:

- Courses
- Faculty
- Students

Step 8 — Attendance Form

appAttendance/forms.py

```
from django import forms
from appCourse.models import Course

class AttendanceForm(forms.Form):

    course = forms.ModelChoiceField(
        queryset=Course.objects.all()
    )
```

Step 9 — Create Templates Folder

Inside project root:

```
templates/
```

Inside:

```
templates/attendance/
```

File:

```
mark_attendance.html
```

Step 10 — Add Template DIR

[settings.py](#)

```
'DIRS': [BASE_DIR / 'templates'],
```

Step 11 — Attendance View

appAttendance/views.py

```
from django.shortcuts import render
from django.shortcuts import redirect

from .forms import AttendanceForm

from appStudent.models import Student
from appAttendance.models import Attendance

from datetime import date

def mark_attendance(request):

    students = []

    if request.method == "POST":

        course_id = request.POST.get("course")

        students = Student.objects.filter(
            course_id=course_id
        )

        selected_students = request.POST.getlist(
            "students"
        )

        if selected_students:

            for sid in selected_students:

                Attendance.objects.get_or_create(

                    student_id=sid,
```



```
        course_id=course_id,

        date=date.today(),

        defaults={

            'faculty_id': 1,
            'present': True
        }
    )

    return redirect('/attendance/')

form = AttendanceForm()

return render(

    request,

    'attendance/mark_attendance.html',

    {
        'form': form,
        'students': students
    }
)
```

Step 12 — URLs

appAttendance/urls.py

```
from django.urls import path
from .views import mark_attendance

urlpatterns = [

    path(
        "",
        mark_attendance
```

```
),  
]
```

Main [urls.py](#)

```
from django.contrib import admin  
from django.urls import path  
from django.urls import include  
  
urlpatterns = [  
  
    path('admin/', admin.site.urls),  
  
    path(  
        'attendance/',  
        include('appAttendance.urls')  
    ),  
]
```

Step 13 — Attendance Template

templates/attendance/mark_attendance.htm

```
<!DOCTYPE html>  
<html>  
  
<head>  
  
    <title>  
        Attendance  
    </title>  
  
    <style>  
  
        .student-container{  
  
            display:flex;
```

```
    flex-wrap:wrap;
    gap:20px;

    margin-top:20px;
}

.student-circle{

    width:80px;
    height:80px;

    border-radius:50%;

    background:red;
    color:white;

    display:flex;
    justify-content:center;
    align-items:center;

    cursor:pointer;

    font-size:20px;
}

.present{

    background:green;
}

</style>

</head>

<body>

    <h1>
        Mark Attendance
    </h1>

    <form method="POST">

        {% csrf_token %}

        {{ form.course }}

        <button type="submit">
```

```
        Load Students
    </button>

    <div class="student-container">

        {% for student in students %}

            <div
                class="student-circle"

                onclick="toggleAttendance(this)"
            >

                {{ student.id }}

                <input
                    type="checkbox"

                    name="students"

                    value="{{ student.id }}"

                    hidden
                >

            </div>

        {% endfor %}

    </div>

    <br>

    {% if students %}

        <button type="submit">

            Mark Attendance

        </button>

    {% endif %}

</form>

<script>
```

```
function toggleAttendance(el){  
  
    el.classList.toggle("present");  
  
    let checkbox = el.querySelector("input");  
  
    checkbox.checked = !checkbox.checked;  
}  
  
</script>  
  
</body>  
  
</html>
```

Result

You now have:

- Project
- Apps
- Models
- Admin panel
- Attendance form
- Circular attendance UI
- Attendance save logic
- One attendance per day restriction

Next Step After This

Next you should build:

1. Faculty Login
2. Student Login
3. Subject Model
4. Attendance Reports
5. Attendance Percentage
6. Bootstrap UI
7. AJAX attendance
8. Charts
9. Search/filter

